TASK 10: Enforce Encapsulation through Packages and Access Modifiers

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Package structure (the planned design)

Our submitted code is in one file for simple compiling on OnlineGDB. The package structure below is the design for a real project. Each package uses a separate folder.

Package	Classes
parking.model	Vehicle, TwoWheeler, Car, ElectricCar, EBike, DeliveryVan, ParkingTicket, Chargeable
parking.pass	Pass, MonthlyPass, DayPass
parking.service	ParkingFacility (allocation, exit and fee logic)
parking.app	Main program and the test cases

Access modifier for each field and method

Item	Modifier	Why
Vehicle.regNo	private final	Only the constructor sets it; nothing else can change it
Vehicle.ratePerHour()	protected abstract	Subclasses (and the same package) must implement it; outside code should not call it directly
Vehicle.calculateFee()	public	The fee is the public result that the facility and tests use
ParkingTicket fields	private	The ticket is a record that must not be edited after issue
ParkingFacility.enter(), exit()	public	The operations the attendant uses
ParkingFacility.slots, active	private	Only the facility may change slot state, so a slot cannot be corrupted from outside
ParkingFacility.createVehicle()	static (package-private in the planned design)	A helper used only inside the service package

Validated getters and setters

Our code validates in the constructor and in the methods. The constructor rejects a blank registration number. calculateFee() rejects negative minutes. chargingFee() rejects negative units. exit() rejects an exit time earlier than the entry time, and release() rejects an invalid floor or slot. Validation happens before any change, so a rejected call leaves the object unchanged.

```
protected Vehicle(String regNo) {
    if (regNo == null || regNo.isBlank())
        throw new IllegalArgumentException("Registration number required");
    this.regNo = regNo;
}
```

Wrapper classes and autoboxing: the risk

Map uses String keys, which are compared with equals() and are safe. Where an int is stored in a collection, Java boxes it to an Integer. Two Integer objects with the same value can be different objects, so == may return false even when the numbers match, and a null Integer causes a NullPointerException when unboxed. For example, Integer a = 1000, b = 1000; then a == b is false, while a.equals(b) is true. The fix is to compare wrapper values with equals() or to use the primitive int wherever possible.

Alternative: public fields with validation in the caller

If fields were public and the calling code checked each value, a caller could set entryTime = -5 or slot = 99 and the object would be broken without any error. Every caller would need the same check, so the rule would be copied in each place it is used (for example, in the facility, the tests, and any future screen), and one forgotten copy is enough to let bad data in. Validation in the object costs O(1) per update (one comparison), the same as in the caller, but it is written once. With n updates from m callers, the caller design needs the check in m places, while the encapsulated design needs it in one place, so a change to the rule is made once and cannot be missed.